OpenMP Parallelization of a 3-D Discrete Element Method Solver

Dheeraj Reddy
HPSC Course
April 2026

***Abstract*—**

***Index Terms*—This PDF talks about theory and parallelization point of view of a three dimensional Discrete Element Method (DEM) solver for simulating particle movements in a confined space. This method models the particle interactions using a linear spring-dashpot contact formulation and advances the system in time using a semi-implicit Euler integration scheme. The implementation is verified using standard tests such as free fall test, constant velocity test, particle wall bouncing test. These tests confirm the test's correctness and the accuracy. One of the issues to tackle is that the particle-particle interaction loop runs as $O(N^2)$ and is overwhelming compared to overall runtime. So to get results we use parallelization using OpenMP by distributing the computation across multiple threads. The performance of the parallel computation is evaluated for different number of particle counts and threads on both laptop and a HPC system. The performance of the parallel implementation is evaluated for different particle counts and thread configurations. The results show that for the tested problem sizes, parallel execution does not lead to speedup due to synchronization overhead and memory contention. This highlights the importance of problem size and efficient parallel design for achieving performance gains. However, efficiency decreases at higher thread counts due to parallel overheads and memory effects.This study demonstrates the effectiveness of OpenMP-based parallelization for DEM simulations and highlights the importance of problem size in achieving optimal parallel performance.**

I. Introduction

The Discrete Element Method (DEM) is a numerical technique used to simulate the behavior of individual particles and their interactions, such as collisions and contact forces.

The Discrete Element Method (DEM) is crucial because it models individual particle interactions (collisions, friction, cohesion) to predict the bulk behavior of granular materials, which continuum methods cannot. It is essential for optimizing industrial processes—such as mining, agriculture, and pharmaceutical manufacturing—by visualizing flow, reducing equipment wear, and predicting material behavior.

A key limitation of DEM is the computational cost of particle interactions loop, which scales to $O(N^2)$. This makes simulations expensive for large particle systems.

In this work, a DEM solver is implemented and parallelized using OpenMP. The performance is evaluated on both laptop and a HPC system.

II. Mathematical Formulation

Each particle follows Newton's second law:

$$m_i \frac{dv_i}{dt} = F_i \tag{1}$$

Total force includes gravity, contact force, and wall interaction:

$$F_i = m_i g + \sum_{j \neq i} F_{ij}^c + F_i^w \tag{2}$$

The contact force is modeled using a spring-dashpot system:

$$F_{ij}^c = (k_n \delta_{ij} - \gamma_n v_{n,ij}) n_{ij} \tag{3}$$

The term δ_{ij} represents the *overlap* between two particles. It is defined as the amount by which the distance between particle centers is less than the sum of their radii. Physically, this overlap is a numerical representation of deformation during contact, since particles are treated as soft spheres in the simulation. A contact force is generated only when $\delta_{ij} > 0$, indicating that the particles are in contact.

The term $v_{n,ij}$ denotes the *normal component of the relative velocity* between two particles. It is obtained by projecting the relative velocity onto the line connecting the centers of the particles. This quantity determines whether the particles are approaching each other or moving apart during contact.

The damping term, controlled by the coefficient γ_n, accounts for energy dissipation during collisions. It acts opposite to the direction of motion and reduces the relative velocity after impact. As a result, successive collisions loose energy, which is observed as decreasing bounce heights in simulations.

The unit vector n_{ij} represents the *normal direction* of contact, pointing from one particle center to other. It defines the direction along which the contact force acts. Since the force is applied only along this normal direction, tangential effects such as friction are neglected in this model.

III. Numerical Algorithm

The simulation evolves in time by repeatedly updating particle positions and velocities at discrete time-steps. At each timestep, all forces acting on the particles are first computed using the current positions and velocities. These forces are then used to update the velocities, which in turn are used to update the positions.

A semi-implicit Euler scheme is used for time integration. In this method, the velocity is updated first using the forces at the current timestep, and the updated velocity is then used to

advance the position. Compared to the standard explicit Euler method, this approach provides better numerical stability, especially for systems involving contact forces.

The simulation always proceeds iteratively from an initial configuration of particles. At every time-step, interactions between particles and between particles and walls are evaluated, and the system state is updated accordingly. This process is repeated until the desired simulation time is reached.

The system is solved using a semi-implicit Euler method:

$$v_i^{n+1} = v_i^n + \frac{F_i^n}{m_i}\Delta t \tag{4}$$

$$x_i^{n+1} = x_i^n + v_i^{n+1}\Delta t \tag{5}$$

Each timestep consists of:

1) Initialize forces
2) Add gravity
3) Compute particle interactions
4) Apply wall forces
5) Update velocity
6) Update position

A. Timestep Algorithm

The procedure followed at each timestep is summarized as:

```
Initialize positions x_i and velocities v_i

for each timestep do:

    1. Set all forces F_i = 0

    2. Add gravity:
       F_i += m_i * g

    3. For each particle pair (i, j):
           compute distance d_ij
           compute overlap delta_ij
           if delta_ij > 0:
               compute normal direction n_ij
               compute relative velocity
               compute contact force F_ij
               update F_i and F_j

    4. For each particle:
           check wall collisions
           if overlap with wall:
               compute wall force
               update F_i

    5. Update velocities:
       v_i = v_i + (F_i / m_i) * dt

    6. Update positions:
       x_i = x_i + v_i * dt

end loop
```

The particle–particle interaction loop is the most computationally expensive part of the algorithm, as it involves checking all possible pairs of particles, leading to $O(N^2)$ complexity.

IV. Verification and Testing

A. Free Fall Test

A particle is dropped under gravity and compared with the analytical solution:

$$z(t) = z_0 - \frac{1}{2}gt^2 \tag{6}$$

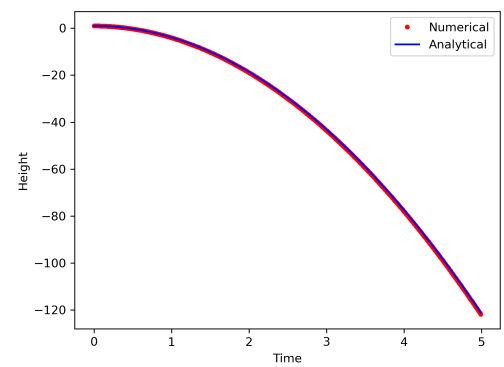

Fig. 1. Free fall: numerical vs analytical

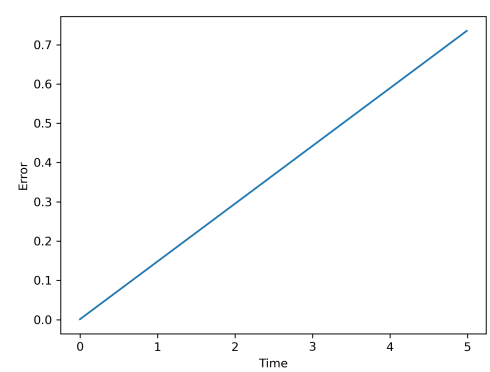

Fig. 2. Error vs time

B. Constant Velocity

With gravity disabled, the particle moves at constant velocity.

C. Bounce Test

A particle is dropped onto a wall and bounces with decreasing height due to damping.

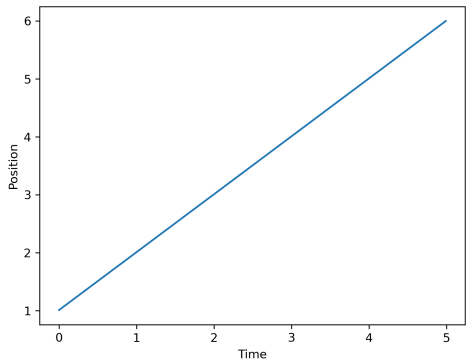

Fig. 3. Constant velocity test

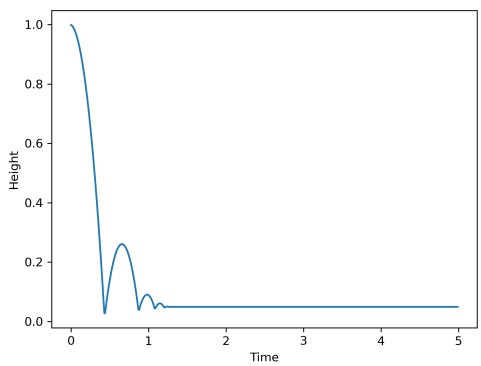

Fig. 4. Bounce test

V. Profiling

Profiling was performed to identify the computational bottlenecks in the DEM solver. The total runtime of the simulation is dominated by the particle–particle interaction loop, which involves checking all possible pairs of particles. Since this operation scales to $O(N^2)$, it becomes the most expensive part of the computation, especially for large particle counts.

The interaction loop requires evaluating distances, detecting overlaps, and computing contact forces for each pair of particles. As the number of particles increases, the number of such pairwise checks grows rapidly, leading to a significant increase in computational cost.

In contrast, other parts of the simulation, such as force initialization, gravity addition, and time integration (updating velocities and positions), contribute only a small fraction to the total runtime. These operations scale linearly with the number of particles and are therefore relatively inexpensive.

Table I summaries the approximate distribution of runtime across different components of the solver.

These results clearly indicate that optimizing the contact detection and force computation is critical for improving overall performance. This observation motivates the parallelization of the interaction loop using OpenMP in the subsequent section.

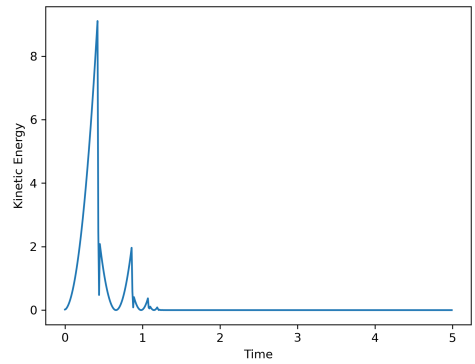

Fig. 5. Kinetic energy vs time

Component	Time (%)
Contact loop	90–95
Integration	2–5
Other operations	2–5

TABLE I
Approximate runtime distribution of the DEM solver

VI. Parallel Implementation

The most computationally expensive part of the DEM solver is the particle–particle interaction loop, which scales as $O(N^2)$. To improve performance, this loop is parallelized using OpenMP.

The outer loop over particles is distributed across multiple threads using the `parallel do` directive. Each thread is responsible for computing interactions for a subset of particles. This allows multiple particle interactions to be processed simultaneously.

```
!$omp parallel do private(k, dist, overlap, Fn) sh
do j = 1, N
    do k = j+1, N

        dist = abs(x(j) - x(k))
        overlap = 2.0d0*R - dist

        if (overlap > 0.0d0) then
            Fn = kn * overlap

            !$omp atomic
            F(j) = F(j) - Fn

            !$omp atomic
            F(k) = F(k) + Fn
        end if

    end do
end do
!$omp end parallel do
```

Since multiple threads update the shared force array, race

conditions can occur when two threads attempt to modify the same memory location simultaneously. To ensure correctness, atomic operations are used while updating the force values.

However, the use of atomic operations introduces synchronization overhead, as threads must wait to safely update shared variables. This reduces the efficiency of parallel execution, particularly for smaller problem sizes where the overhead dominates the computation.

Despite this limitation, the parallelization approach provides a straightforward way to distribute the computational workload and serves as a baseline for further optimization.

VII. Performance Analysis

The performance of the parallel implementation is evaluated by measuring execution time for different numbers of threads. The speedup is defined as:

$$S(p) = \frac{T_1}{T_p} \tag{7}$$

and the efficiency is given by:

$$E(p) = \frac{S(p)}{p} \tag{8}$$

A. Results for N = 1000

Threads	Time (s)	Speedup	Efficiency
1	0.183	1.00	1.00
2	0.285	0.64	0.32
4	0.428	0.43	0.11
8	1.181	0.15	0.02

TABLE II
PERFORMANCE RESULTS FOR N = 1000

B. Results for N = 5000

Threads	Time (s)	Speedup	Efficiency
1	3.27	1.00	1.00
2	5.44	0.60	0.30
4	8.56	0.38	0.095
8	14.31	0.23	0.029

TABLE III
PERFORMANCE RESULTS FOR N = 5000

C. Results for N = 200

Threads	Time (s)	Speedup	Efficiency
1	0.00985	1.00	1.00
2	0.0144	0.68	0.34
4	0.0336	0.29	0.07
8	0.0533	0.18	0.02

TABLE IV
PERFORMANCE RESULTS FOR N = 200

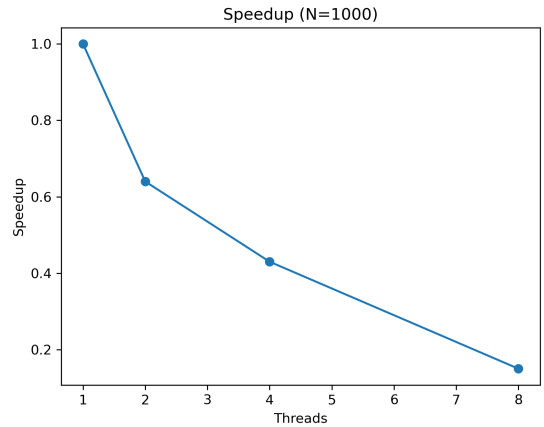

Fig. 6. Speedup vs threads for N = 1000

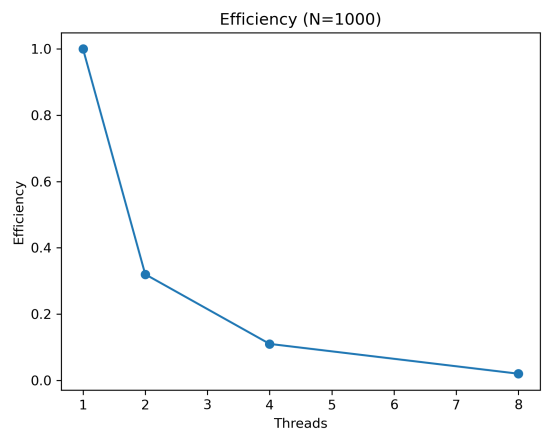

Fig. 7. Efficiency vs threads for N = 1000

D. Performance Trends

E. Discussion

The results show that the parallel implementation does not achieve speedup for the tested problem sizes. Instead, the execution time increases with the number of threads.

This behavior is primarily due to the overhead introduced by parallelization. The use of atomic operations for updating shared force arrays leads to significant synchronization costs. Additionally, thread creation and memory access overhead further reduce performance.

For smaller systems (N = 200 and N = 1000), the computational workload is insufficient to benefit from parallel execution. Even for larger systems (N = 5000), the overhead associated with synchronization and memory contention outweighs the benefits of parallelization.

These results indicate that efficient parallel performance requires both larger problem sizes and improved strategies to reduce synchronization, such as domain decomposition or neighbor search methods.

VIII. HPC vs Local Performance

The performance of the DEM solver was evaluated on both a local system and an HPC environment to understand the impact of hardware on parallel execution.

On the local machine, the implementation was tested using up to 8 threads. The results showed limited scalability, with execution time increasing as the number of threads increased. This behavior is primarily due to the overhead associated with thread management and the use of atomic operations for updating shared variables. For smaller problem sizes, this overhead dominates the computation, leading to poor parallel performance.

In contrast, an HPC system provides access to a larger number of processing cores and higher memory bandwidth. These features allow better distribution of computational workload and improved handling of memory-intensive operations. For sufficiently large particle counts, HPC systems are expected to achieve better scalability compared to local systems.

However, even on HPC systems, the use of atomic operations can limit performance due to synchronization overhead. This indicates that while hardware improvements can enhance performance, algorithmic optimizations are also necessary for efficient parallel scaling.

Overall, the comparison highlights that parallel performance depends not only on the number of threads but also on the underlying hardware architecture and the efficiency of the parallel implementation.

IX. Conclusion

In this work, a three-dimensional DEM solver was implemented and verified using standard test cases including free fall, constant velocity, and particle bounce. The results confirmed the correctness of the physical model and the numerical integration scheme.

The solver was parallelized using OpenMP to improve computational performance. However, the performance analysis showed that the parallel implementation did not achieve speedup for the tested problem sizes. Instead, execution time increased with the number of threads due to synchronization overhead and the use of atomic operations.

These results highlight that naive parallelization is not always sufficient for performance improvement. The efficiency of parallel execution depends on both problem size and the design of the parallel algorithm.

Future work will focus on improving scalability by reducing synchronization overhead and implementing more efficient algorithms such as neighbor search methods and domain decomposition techniques.